The WIMS 1.0 water infiltration system

User manual

Introduction

The WIMS 1.0 is the water infiltration measurement system produced and sold by Servizi Territorio srl.

It is composed by one to five single-ring infiltrometers, each equipped with a Mariotte bottle allowing both visual and automatic level recording, and a data acquisition and storage system.

The whole system is battery operated, using two rechargeable 3.6V Li-ion 18650 batteries.

In Figure 1 one can see the general aspect of the data acquisition system, and in Figure 2 an infiltrometer + Mariotte kit is shown.

The operating principle is simple: once the infiltrometer has been positioned, the Mariotte bottle is placed over it and levelled, and then filled with water. The water then outflows, maintaining a constant level in the infiltrometer. The decrease of water in the Mariotte bottle can be both visually determined thanks to the bottle's graduated scale, and read automatically using a differential pressure sensor.

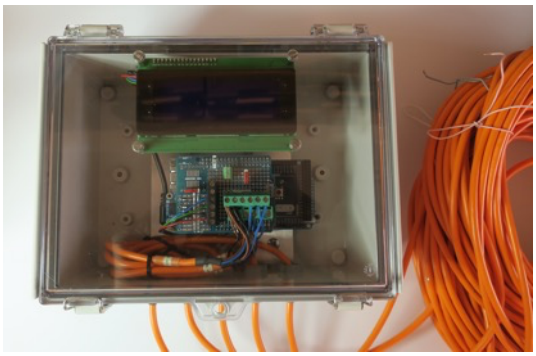	
Figure 1 – *Data acquisition and recording system*	Figure 2 – *Mariotte bottle and its single-ring infiltrometer*

Positioning the infiltrometer and its Mariotte bottle

The single-ring infiltrometer is placed over the measurement area and inserted after proper preparation of the ground. Water is then poured into it, and the Mariotte bottle placed immediately over as shown in Figure 3.

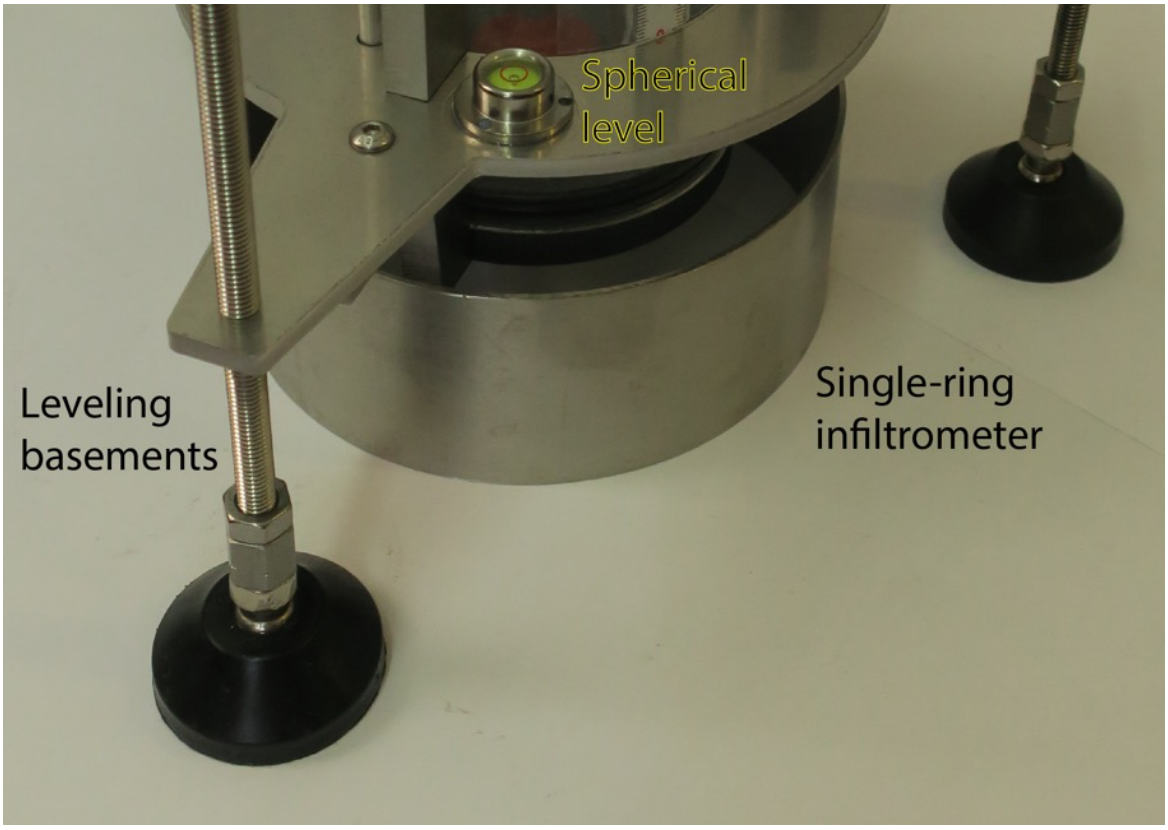

Figure 3 – *Mariotte bottle positioned over the infiltrometer, and levelled using the three threaded levelling basements*

Connecting the Mariotte bottles differential pressure sensors to the data acquisition and recording unit

As shown in Figure 4, all cables from the data acquisition and measurement unit are labelled with the channel number, from 0 to 4. The cable connector with label "*n*" should then be connected to the corresponding pressure measuring module with same number, indicated in Figures 5 and 6.

Before to operate the system please ensure all relevant connectors are firmly fixed, and cables do not interfere with people passage.

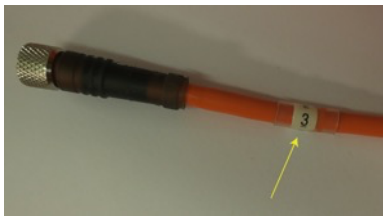		
Figure 4 – *Cable connector (on left) and its channel label (indicated by arrow)*	Figure 5 – *Differential pressure measurement module: the channel number can be seen on the right to the serial number*	Figure 6 – *Detail of the connector on differential pressure: cable connector with same channel number is connected here*

Using the data acquisition and recording unit

Power supply

The system is operated by battery, using two 3.6V Li-Ion 18650 units placed in the holder shown in Figure 7, following the alignment shown in Figure 8.

Figure 7 – *Detail of the battery holder; finger shows how to open*	Figure 8 – *Battery holder shown in open position: notice the +/- alignment*

At least 8 hours can be safely expected if fully charged 2500 mAh batteries are used, the LCD is not excluded (i.e. lighted and operational), all Mariotte bottles are connected, and data acquisition is in progress. The real duration depends on battery capacity and use profile.

User switches

The system is operated through three switches, accessible by opening the system's container.

The switches are:

- Card remove (Figure 9)
- LCD exclude (Figure 9)
- Power switch (Figure 10)

The reason for card remove switch is to help preventing the MicroSD card is removed while the system is writing to it: this event, although occurring with low probability, may cause MicroSD unrecoverable card corruption.

The LCD exclude switch can be used to reduce the power consumption.

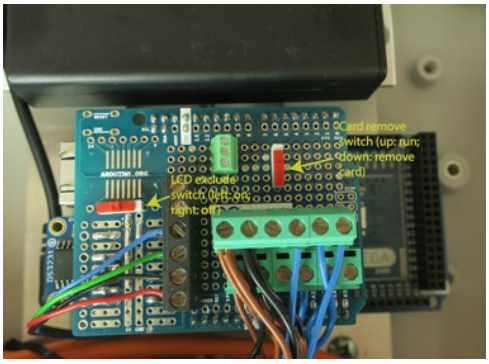

Figure 9 - *Card remove and LCD exclude switches*

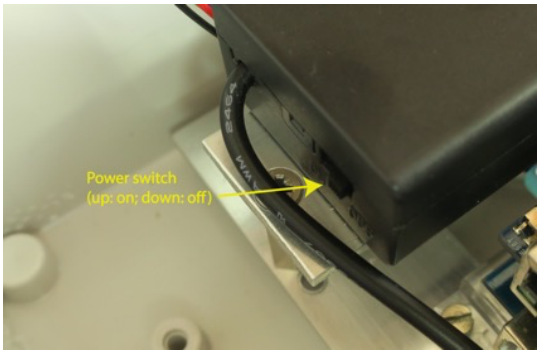

Figure 10 – *Power on-off switch*

System state

The time evolution of the system state is presented in the diagram in Figure 11.

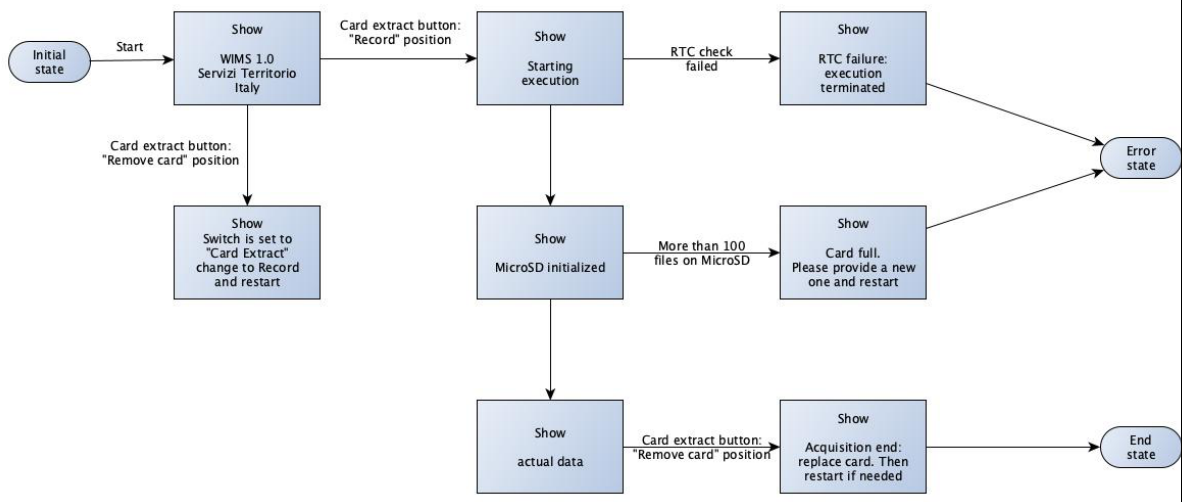

Figure 11 – *State and transitions chard of WIMS 1.0*

File format

Data are written to MicroSD card in the following format:

- Name: **RUN_*nn*.CSV** (***nn*** is an integer from 00 to 99)
- Header: "`tm, A0, A1, A2, A3, A4`"
- Records:
  - Time stamp, ISO form.
  - Channel 0 reading.
  - Channel 1 reading.
  - Channel 2 reading.
  - Channel 3 reading.
  - Channel 4 reading.

Arduino sketch

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SD.h>
#include <RTClib.h>

// Real-time clock chip is a DS3231 high-precision, battery-backed
RTC_DS3231 rtc;

// The LCD is quite "generic"
LiquidCrystal_I2C lcd(0x27,20,4);

// SD card related
const int chipSelect = 4;
File dataFile;
char filename[] = "RUN_00.CSV";

// Pins
#define RESET   8
#define SWITCH A5

// Switch value corresponding to "recording" mode (change if needed)
#define RECORD 500

// Data recording state
bool bRecordOld;
bool bRecord;

// Time management through RTC
bool bFirstMeasurement;
long int startingTime;



void setup() {

  // Configure pushbutton input
```

```
pinMode(SWITCH, INPUT);
pinMode(RESET, OUTPUT);
digitalWrite(RESET, HIGH);

while(!Serial);
Serial.begin(115200);

// Start and initialize RTC
Wire.begin();
Serial.println("Wire initialized");

// Initial state is "recording", whatever the pushbutton mechanical state
bRecordOld = true;
bRecord = (analogRead(SWITCH) > RECORD);
Serial.print("Switch set to ");
if(bRecord) Serial.println("Record mode");
else Serial.println("Card extract mode");

lcd.init();       //initialize the lcd
lcd.backlight();  //open the backlight
lcd.setCursor(6, 0);
lcd.print("WIMS 1.0");
lcd.setCursor(0, 2);
lcd.print(" Servizi Territorio");
lcd.setCursor(6, 3);
lcd.print("(Italy)");
delay(3000);

// Check initial switch state to be RECORD
lcd.init();
Serial.println("LCD initialized");
lcd.backlight();
if(analogRead(SWITCH) > RECORD) {
  lcd.setCursor(0, 0);
  lcd.print("Starting execution");
}
else {
  lcd.setCursor(0, 0);
  lcd.print("Switch is set to");
  lcd.setCursor(0, 1);
  lcd.print("'Card extract':");
  lcd.setCursor(0, 2);
  lcd.print("change to Record");
  lcd.setCursor(0, 3);
  lcd.print("and restart");
  while (1) delay(10);
}
Serial.println("LCD: message 1 written");

if (!rtc.begin()) {
  lcd.init();       //initialize the lcd
```

```
  lcd.backlight();  //open the backlight
  lcd.setCursor(0, 0);
  lcd.print("RTC failure:");
  lcd.setCursor(0, 1);
  lcd.print("execution");
  lcd.setCursor(0, 2);
  lcd.print("terminated");
  Serial.println("Failure initializing the RTC");
  while (1) delay(10);
}
Serial.println("LCD: message 2 written");
/*if (!rtc.lostPower()) {
  // Ops! The RTC lost power since last activation
  lcd.init();       //initialize the lcd
  lcd.backlight();  //open the backlight
  lcd.setCursor(0, 0);
  lcd.print("RTC lost power");
  lcd.setCursor(0, 1);
  lcd.print("Please check");
  lcd.setCursor(0, 2);
  lcd.print("RTC battery.");
  lcd.setCursor(0, 3);
  lcd.print("Zeroing time.");
  rtc.adjust(DateTime(1970, 1, 1, 0, 0, 0));
}*/

// Connect card, and check the MicroSD is inserted
pinMode(chipSelect, OUTPUT);
if (!SD.begin(chipSelect)) {
  Serial.println("Card failed");
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Card missing or");
  lcd.setCursor(0, 1);
  lcd.print("faulty.");
  lcd.setCursor(0, 2);
  lcd.print("Please provide a new");
  lcd.setCursor(0, 3);
  lcd.print("one and restart");
  while(true) delay(10);
}
Serial.println("Card initialized.");
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("MicroSD initialized.");
delay(1000);

// Create a new file
for (int i = 0; i <= 100; i++) {
  if(i >= 100) {
    lcd.clear();
```

```
      lcd.setCursor(0, 0);
      lcd.print("Card full.");
      lcd.setCursor(0, 1);
      lcd.print("Please provide a new");
      lcd.setCursor(0, 2);
      lcd.print("one and restart.");
      while(true) delay(10);
    }
    filename[4] = i / 10 + '0';
    filename[5] = i % 10 + '0';
    if (! SD.exists(filename)) {
      // only open a new file if it doesn't exist
      dataFile = SD.open(filename, FILE_WRITE);
      dataFile.print("Date");
      dataFile.print(",");
      dataFile.print("time (h:m:s)");
      dataFile.print(",");
      dataFile.print("time (s)");
      dataFile.print(",");
      dataFile.print("P0 (Volt)");
      dataFile.print(",");
      dataFile.print("P1 (Volt)");
      dataFile.print(",");
      dataFile.print("P2 (Volt)");
      dataFile.print(",");
      dataFile.print("P3 (Volt)");
      dataFile.print(",");
      dataFile.println("P4 (Volt)");
      break; // leave the loop!
    }
  }
  Serial.println(filename);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Writing to file");
  lcd.setCursor(0, 1);
  lcd.print(filename);
  delay(2000);

  // Set resolution to optimal value (found during
  // analog input channels characterization)
  //analogReadResolution(10);
  Serial.println("tm, A0, A1, A2, A3, A4");

  // Notify the next measurement is the first one
  bFirstMeasurement = true;

}


void loop() {
```

```
// Set initial time, if needed
long int currentTime;
DateTime now;
if(bFirstMeasurement) {
  now = rtc.now().unixtime();
  startingTime = now.unixtime();
  currentTime = startingTime;
  bFirstMeasurement = false;
}
else {
  now = rtc.now().unixtime();
  currentTime = now.unixtime();
}
long int deltaTime = currentTime - startingTime;

// Take channel readings
float a0 = analogRead(A0) / 1024.0f * 5.f;
float a1 = analogRead(A1) / 1024.0f * 5.f;
float a2 = analogRead(A2) / 1024.0f * 5.f;
float a3 = analogRead(A3) / 1024.0f * 5.f;
float a4 = analogRead(A4) / 1024.0f * 5.f;

// Update state based on switch, and act depending on transition
bRecordOld = bRecord;
bRecord = analogRead(SWITCH) > RECORD;
if(bRecord && bRecordOld) {

  // Do nothing special:
  // "business as usual"

}
else if(bRecord && !bRecordOld) {

  // Error message (will never happen really: trapped before)
  lcd.init();
  lcd.setCursor(0, 0);
  lcd.print("File already closed");
  lcd.setCursor(0, 1);
  lcd.print("Please restart");
  while(true) delay(10);

}
else if(!bRecord) {

  // Close file
  dataFile.close();

  // Tell users the card may be removed
  lcd.init();
  lcd.setCursor(0, 0);
```

```
  lcd.print("Acquisition end:");
  lcd.setCursor(0, 1);
  lcd.print("replace card. Then");
  lcd.setCursor(0, 2);
  lcd.print("restart if needed");
  while(true) delay(10);

}
else {
  // Just refresh status indications
}

// Data, to serial
Serial.print(millis() / 1000.0f, 2);
Serial.print(", "); Serial.print(a0);
Serial.print(", "); Serial.print(a1);
Serial.print(", "); Serial.print(a2);
Serial.print(", "); Serial.print(a3);
Serial.print(", "); Serial.print(a4);
Serial.println();

// Data, to MicroSD
int n_bytes = 0;
n_bytes += dataFile.print(now.year(), DEC);
n_bytes += dataFile.print("/");
n_bytes += dataFile.print(now.month(), DEC);
n_bytes += dataFile.print("/");
n_bytes += dataFile.print(now.day(), DEC);
n_bytes += dataFile.print(",");
n_bytes += dataFile.print(now.hour(), DEC);
n_bytes += dataFile.print(":");
n_bytes += dataFile.print(now.minute(), DEC);
n_bytes += dataFile.print(":");
n_bytes += dataFile.print(now.second(), DEC);
n_bytes += dataFile.print(",");
n_bytes += dataFile.print(deltaTime);
n_bytes += dataFile.print(",");
n_bytes += dataFile.print(a0, 3);
n_bytes += dataFile.print(",");
n_bytes += dataFile.print(a1, 3);
n_bytes += dataFile.print(",");
n_bytes += dataFile.print(a2, 3);
n_bytes += dataFile.print(",");
n_bytes += dataFile.print(a3, 3);
n_bytes += dataFile.print(",");
n_bytes += dataFile.println(a4, 3);
dataFile.flush();

// Check if no bytes have been written
if(n_bytes <= 0) {
  lcd.init();
```

```
    lcd.setCursor(0, 0);
    lcd.print("Card prematurely");
    lcd.setCursor(0, 1);
    lcd.print("removed or failed.");
    lcd.setCursor(0, 2);
    lcd.print("Please insert a new");
    lcd.setCursor(0, 3);
    lcd.print("one and restart");
    while(true) delay(10);
  }

  // Print data to LCD
  char sMessage[32];
  lcd.init();
  sprintf(sMessage, "T:%7d %s", deltaTime, filename);
  lcd.setCursor(0, 0);
  lcd.print(sMessage);
  lcd.setCursor(0, 1);
  lcd.print("0:");
  lcd.print(a0);
  lcd.print(" 1:");
  lcd.print(a1);
  lcd.print(" 2:");
  lcd.print(a2);
  lcd.setCursor(0, 2);
  lcd.print("3:");
  lcd.print(a3);
  lcd.print(" 4:");
  lcd.print(a4);
  if(bRecord) {
    sprintf(sMessage, "B:%2.2d Recording", n_bytes);
    lcd.setCursor(0, 3);
    lcd.print(sMessage);
  }
  else {
    // Defensive: this part should never be entered, being trapped before
    sprintf(sMessage, "B:%2.2d Idle", n_bytes);
    lcd.setCursor(0, 3);
    lcd.print(sMessage);
  }

  delay(200);

}
```

Resetting date and time

Resend script to Arduino (Mega).